

网络技术与应用实验报告

实验5：简单路由器程序的设计

姓名：谢小珂 学号：2310422 专业：物联网工程

一、实验要求

简单路由器程序设计实验的具体要求为：

- （1）设计和实现一个路由器程序，要求完成的路由器程序能和现有的路由器产品（如思科路由器、华为路由器、微软的路由器等）进行协同工作。
- （2）程序可以仅实现IP数据报的获取、选路、投递等路由器要求的基本功能。可以忽略分片处理、选项处理、动态路由表生成等功能。
- （3）需要给出路由表的手工插入、删除方法。
- （4）需要给出路由器的工作日志，显示数据报获取和转发过程。
- （5）完成的程序须通过现场测试，并在班（或小组）中展示和报告自己的设计思路、开发和实现过程、测试方法和过程。

二、实验原理

2.1 路由器的基本功能

路由器工作在 OSI 参考模型的网络层，核心功能是根据目的 IP 地址和路由表，对收到的 IP 数据报进行选路和转发。每台路由器维护一张路由表，记录各个目标网络的目的网络地址、子网掩码以及下一跳地址或出接口。当路由器收到 IP 数据报时，会根据报文中的目的 IP，在路由表中执行“最长前缀匹配”，找到与该目的地址匹配度最高的一条路由，然后按照该路由项给出的下一跳信息将数据报转发出去。

2.2 IP 数据报在链路上的封装与转发

在以太网中真实传输的是以太网帧，IP 数据报作为负载被封装在以太网帧内部。发送端主机首先根据自己的 IP 地址和子网掩码判断目的 IP 是否与自己处于同一网段：

- 若目的主机与自己处于同一网段，则主机通过 ARP 协议解析出对方主机的 MAC 地址，将以太网帧首部中的目的 MAC 填写为对方主机的 MAC，源 MAC 为本机网卡的 MAC，IP 首部中的源 IP、目的 IP 分别为通信双方的 IP；
- 若目的主机不在本地网段，则主机将默认网关（路由器接口）作为下一跳，通过 ARP 协议解析出网关接口的 MAC 地址，将以太网帧的目的 MAC 填写为网关的 MAC 地址，而 IP 首部中的目的 IP 仍然保持为最终目的主机的 IP 地址。

IP 数据报到达路由器后，路由器只查看 IP 首部中的目的 IP 地址进行路由选择，不关心上层传输层及应用层数据。典型处理步骤为：

- 1) 检查 IP 首部校验和是否正确，若错误则丢弃；
- 2) 检查并递减 TTL 字段，TTL 减 1 后重新计算 IP 首部校验和；
- 3) 根据目的 IP 地址在路由表中进行最长前缀匹配，得到下一跳 IP 地址（直连网络时下一跳就是目的 IP 本身）；
- 4) 根据下一跳 IP，在 ARP 表中查询其对应的 MAC 地址，若存在则使用该 MAC 封装新的以太网首部；

- 源 MAC：当前路由器出口接口的 MAC；
- 目的 MAC：下一跳设备（下一个路由器或最终主机）的 MAC；
- IP 首部中的源/目的 IP 均保持不变，仅 TTL 递减。

由此可见，IP 数据报从源主机到目的主机的转发过程中，网络层的源/目的 IP 地址保持端到端不变，而链路层的源/目的 MAC 地址在每一跳都会被重新封装。

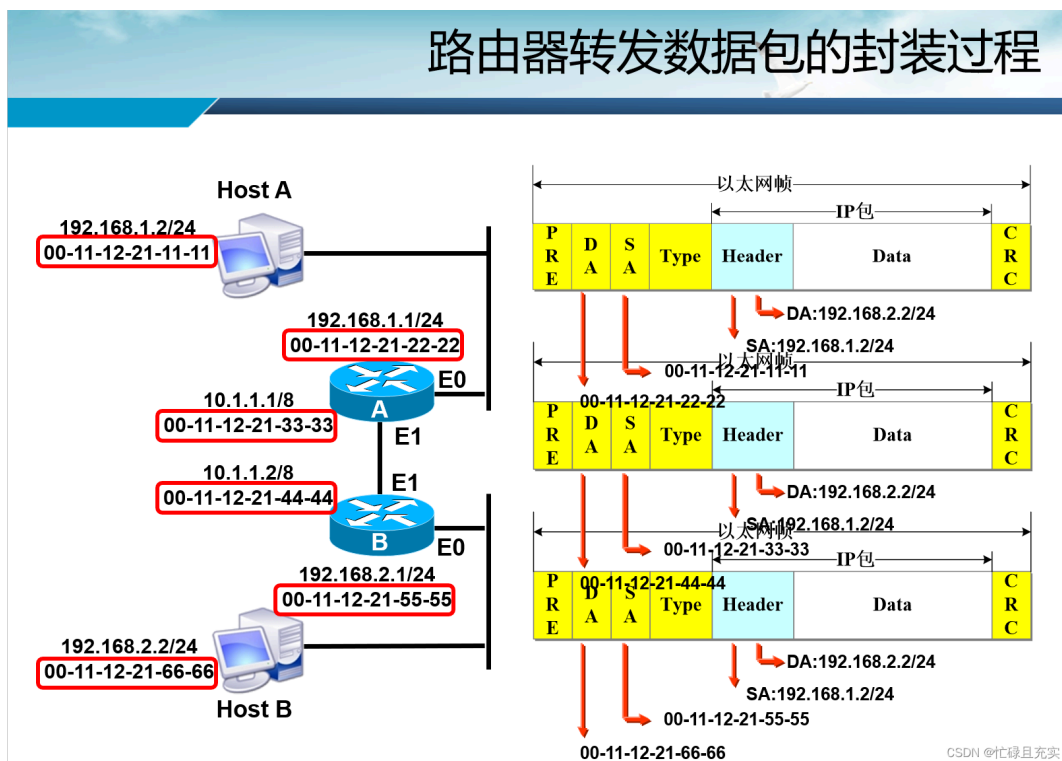


图 1 路由器转发数据包的封装过程

2.3 本实验中软件路由器的工作原理

本实验在 Windows 平台上基于 WinPcap 实现一个软件路由器程序，将其部署在虚拟机 2 上，该虚拟机同一网卡上配置了两个 IP 地址（206.1.1.1 和 206.1.2.1），分别连接到两个不同网段。程序通过 WinPcap 打开网卡，伪造 ARP 报文获取本机 MAC 地址，并根据本机两个 IP 及其子网掩码自动生成两条直连路由；同时在内存中维护一张静态路由表 and 一张 ARP 表，以及一个缓存队列用于暂存等待 ARP 解析的 IP 数据报。

路由器程序只接收并处理两类以太网帧：ARP 帧和 IP 帧。其工作流程大致如下：

- 对于 ARP 响应报文：解析出报文中的 IP-MAC 映射关系，更新 ARP 表；同时检查缓存队列中是否有以该 IP 为下一跳的待转发数据报，若有则补全目的 MAC 并立即转发。
- 对于 IP 数据报：首先验证 IP 首部校验和和 TTL 是否有效，丢弃 TTL 小于等于 1 或校验和错误的报文；然后检查目的 IP 是否为本机任一接口地址，若是则不再转发；若需要转发，则在路由表中执行最长前缀匹配得到下一跳 IP 地址，再在 ARP 表中查找该 IP 对应的 MAC 地址，若查询成功则构造新的以太网首部（源 MAC 为本机出口接口 MAC，目的 MAC 为下一跳 MAC），同时将 TTL 减 1 并重新计算 IP 首部校验和，最后通过 WinPcap 将该帧发送出去；如果 ARP 表中尚无该 IP 对应的 MAC，则先将数据报缓存在缓存队列中，并向该 IP 发送 ARP 请求，待收到 ARP 响应后再转发缓存报文。

通过在主机 1（206.1.1.2）与主机 4（206.1.3.2）之间进行 ping 测试，可以验证软件路由器是否正确实现了 IP 数据报的获取、路由选择和转发过程，并通过程序输出的日志观察每一跳的封装与转发细节。

三、实验过程

3.1 实验环境配置与程序部署

(1) 实验环境概述

本实验在一台安装了 Visual Studio 2022 的 Windows 主机上完成路由器程序的编写与编译，安装了Npcap SDK作为抓包库。在此基础上，教师提供了一个由四台 Windows 虚拟机组成的路由实验环境，本实验主要在**虚拟机 2**上运行自编写的路由器程序，在虚拟机 1、3、4 上进行连通性与转发测试。

(2) 路由器程序的编译与生成

首先在宿主机上使用 Visual Studio 打开 ROUTER 工程，选择“**Release | Win32**”配置，保证生成 32 位可执行文件。点击“生成解决方案”后，在输出窗口中可以看到“生成: 1 成功, 0 失败”的提示信息，表明在 Release Win32 配置下成功完成了编译和链接。

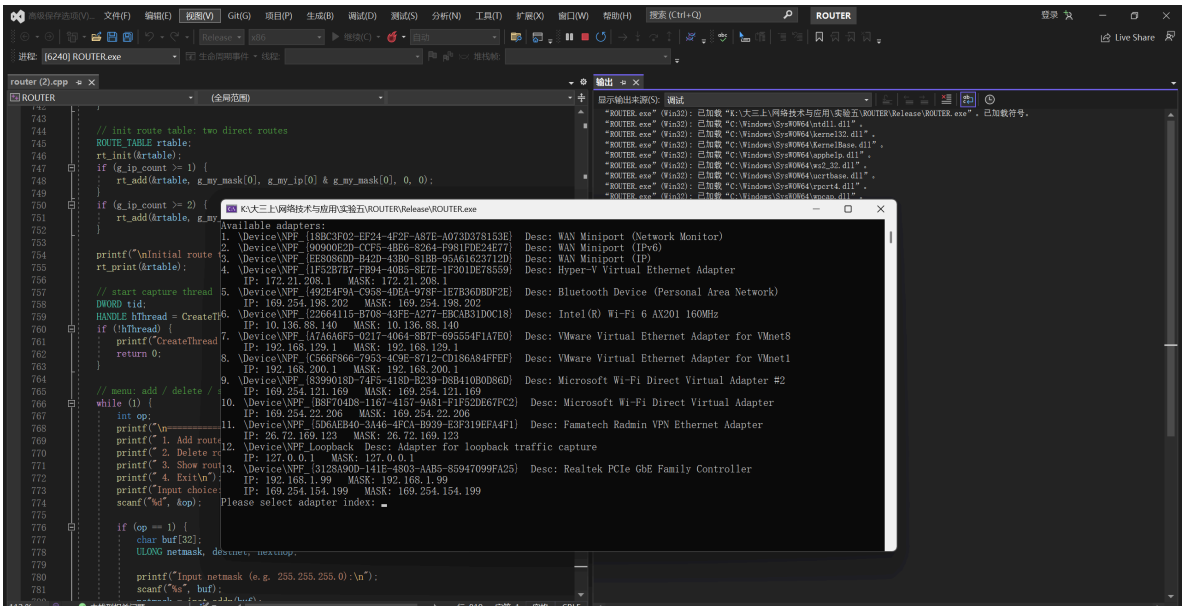


编译完成后，在工程目录下的 ROUTER\Release 文件夹中生成了 ROUTER.exe 及相关中间文件。

	ROUTER.exe	2025/12/8 14:37	应用程序
	ROUTER.exe.recipe	2025/12/8 14:37	RECIPE 文件
	ROUTER.log	2025/12/8 14:37	文本文档
	ROUTER.pdb	2025/12/8 14:37	Program Debug Da...
	ROUTER.intermediate.1	2025/12/8 14:37	尚未完成

(3) 宿主机关上的运行验证与部署到虚拟机

为确认生成的可执行文件工作正常，在宿主机关上直接运行 Release 版本的 ROUTER.exe。程序启动后列出了本机所有可用网络适配器及其 IP / Mask 信息，并提示用户选择适配器序号，说明 WinPcap 抓包接口初始化成功，程序能够正确识别网络接口。



在完成宿主机关上的基本验证后，将 ROUTER\Release 目录中的 ROUTER.exe 拷贝到虚拟机 2 的桌面，作为后续路由转发实验中运行的软件路由器。

3.2 软件路由器程序设计与实现过程

本程序的核心是：

- 用 WinPcap 捕获/发送以太网帧；
- 根据路由表做最长前缀匹配，决定下一跳；
- 利用 ARP 表把“IP → MAC”解析出来；
- 在必要时缓存 IP 数据报，等待 ARP 完成后再转发。

3.2.1 关键数据结构与全局变量

设计目的：

- 用结构体描述以太网帧头、ARP 报文和 IP 首部，便于从原始数据直接强转。
- 用链表保存路由表，用数组保存 ARP 表和缓存队列，结构简单、容易在实验中观察。

```
1  #pragma pack(push, 1) // 1 字节对齐，保证结构体布局和网络报文一致
2  // 以太网帧头
3  typedef struct _ETH_HEADER {
4      BYTE dest[6]; // 目的 MAC
5      BYTE src[6]; // 源 MAC
6      WORD type; // 上层协议类型：0x0800=IP, 0x0806=ARP
7  } ETH_HEADER;
8  // IP 首部（忽略选项字段）
9  typedef struct _IP_HEADER {
10     BYTE ver_ihl; // 高 4 位版本号，低 4 位首部长度
11     BYTE tos;
12     WORD tot_len;
13     WORD id;
14     WORD frag_off;
15     BYTE ttl; // 生存时间
16     BYTE protocol;
17     WORD check; // 首部校验和
18     ULONG saddr; // 源 IP（网络字节序）
19     ULONG daddr; // 目的 IP
20 } IP_HEADER;
21 #pragma pack(pop)
22 // 路由表项：一条路由记录
23 typedef struct _ROUTE_ITEM {
24     ULONG netmask; // 子网掩码
25     ULONG destnet; // 目的网络地址
26     ULONG nexthop; // 下一跳 IP, 0 表示直连
27     int type; // 0=直连路由, 1=手工静态路由
28     struct _ROUTE_ITEM* next; // 单向链表
29 } ROUTE_ITEM;
30 // 路由表头
31 typedef struct _ROUTE_TABLE {
32     ROUTE_ITEM* head;
33     int count;
34 } ROUTE_TABLE;
35 // ARP 表项：记录 IP<->MAC 映射
```

```

36 typedef struct _ARP_ENTRY {
37     ULONG ip;
38     BYTE  mac[6];
39     int   used;
40 } ARP_ENTRY;
41 // 缓存队列项：等待 ARP 解析的数据报
42 typedef struct _BUFFER_ENTRY {
43     BYTE  data[2000];
44     int   len;
45     ULONG dest_ip;    // 需要解析 MAC 的 IP（一般是下一跳 IP）
46     int   used;
47     clock_t ts;       // 入队时间，用于超时控制
48 } BUFFER_ENTRY;
49 // 全局变量（本机两个 IP、掩码和 MAC，ARP 表、缓存队列等）
50 pcap_t* g_adhandle = NULL;
51 BYTE  g_my_mac[6] = { 0 };
52 ULONG g_my_ip[2]  = { 0 };
53 ULONG g_my_mask[2] = { 0 };
54 int   g_ip_count  = 0;
55 ARP_ENTRY  g_arp_table[MAX_ARP];
56 BUFFER_ENTRY g_buffer[MAX_BUFFER];

```

分析：

- 由于所有报文都是“按字节”从网卡抓到的，必须用 `#pragma pack(1)` 保证结构体没有填充字节。
- 路由表用链表是为了方便插入和删除，实验中要动态添加、删除静态路由。
- ARP 表和缓存队列使用定长数组实现，便于实验中打印观察和做超时控制。

3.2.2 路由表实现：自动生成直连路由 + 手工静态路由 + 最长前缀匹配

设计目标：

- 程序启动后根据本机两个接口 IP 自动插入两条直连路由（206.1.1.0/24 和 206.1.2.0/24）；
- 用户通过菜单添加静态路由（例如 206.1.3.0/24 → 206.1.2.2）；
- 查表时按照“最长前缀匹配”原则选择最具体的那一条路由。

(1) 插入路由（区分直连和静态，静态按掩码从大到小）

```

1 // 向路由表插入一条记录
2 // type = 0: 直连路由，始终保留在表头区域
3 // type = 1: 用户添加的静态路由，按 netmask 从大到小插入
4 void rt_add(ROUTE_TABLE* rt,
5             ULONG netmask, ULONG destnet,
6             ULONG nexthop, int type)
7 {
8     ROUTE_ITEM* item = (ROUTE_ITEM*)malloc(sizeof(ROUTE_ITEM));
9     item->netmask = netmask;
10    item->destnet = destnet;
11    item->nexthop = nexthop;
12    item->type    = type;
13    item->next    = NULL;

```

```

14     rt->count++;
15     // 空表，直接做头结点
16     if (rt->head == NULL) {
17         rt->head = item;
18         return;
19     }
20     // 直连路由：插在所有直连项的后面（确保直连在链表前部）
21     if (type == 0) {
22         ROUTE_ITEM* prev = NULL;
23         ROUTE_ITEM* cur = rt->head;
24         while (cur && cur->type == 0) { // 跳过已有直连路由
25             prev = cur;
26             cur = cur->next;
27         }
28         if (prev == NULL) { // 目前没有直连路由
29             item->next = rt->head;
30             rt->head = item;
31         } else {
32             item->next = cur;
33             prev->next = item;
34         }
35         return;
36     }
37
38     // 静态路由：按掩码长短插入，长的在前 => 最长前缀匹配
39     ROUTE_ITEM* prev = NULL;
40     ROUTE_ITEM* cur = rt->head;
41     while (cur && (cur->type == 0 || cur->netmask > netmask)) {
42         prev = cur;
43         cur = cur->next;
44     }
45     if (prev == NULL) { // 插到表头
46         item->next = rt->head;
47         rt->head = item;
48     } else { // 插到 prev 与 cur 中间
49         item->next = cur;
50         prev->next = item;
51     }
52 }

```

(2) 查找下一跳：顺序遍历实现最长前缀匹配

```

1 // 根据目的 IP 查路由表，返回下一跳 IP
2 // 若为直连网络，则返回原始 dest_ip；查不到返回 0xFFFFFFFF
3 ULONG rt_lookup(ROUTE_TABLE* rt, ULONG dest_ip)
4 {
5     ROUTE_ITEM* cur = rt->head;
6     while (cur) { // 匹配条件：目的 IP 与掩码按位与后等于目的网络dest_ip 与 netmask 按位与后 ==
destnet
7         if ((dest_ip & cur->netmask) == cur->destnet) {
8             return (cur->nexthop == 0) ? dest_ip : cur->nexthop; // nexthop = 0 表示直
连路由，下一跳就是对端主机
9         }
10        cur = cur->next;
11    }
12    return 0xFFFFFFFF; //没有匹配的路由
13 }

```

分析：

- 由于静态路由按掩码从大到小插入，只要**从头遍历**，第一个匹配的就是“前缀最长”的那条路由，满足最长前缀匹配要求。
- 若 `nexthop == 0` 表示直连网络，此时下一跳 IP 就是目的 IP；否则使用静态路由中配置的下一跳。
- 实验中添加 `206.1.3.0/24 → 206.1.2.2` 后，发往 206.1.3.x 的报文会先匹配到这条静态路由，并被转发给 VM3。

3.2.3 ARP 表与缓存队列：IP→MAC 解析与等待机制

设计目标：

- ARP 表保存已经解析过的 IP-MAC 映射；
- 若转发时 ARP 表中还没有下一跳的 MAC，要先缓存该 IP 数据报，再主动发送 ARP 请求；
- 收到 ARP 应答后，从缓存队列中把相关数据报取出并转发；
- 超时未解析的缓存数据报要丢弃并记录日志。

(1) ARP 表查找与更新

```

1 // 在 ARP 表中查找 IP 对应的 MAC
2 int arp_find(ULONG ip, BYTE mac[6]) {
3     for (int i = 0; i < MAX_ARP; ++i) {
4         if (g_arp_table[i].used && g_arp_table[i].ip == ip) {
5             memcpy(mac, g_arp_table[i].mac, 6);
6             return 1; // 查找成功
7         }
8     }
9     return 0; // 没找到
10 }
11 // 将一条 IP-MAC 记录加入 ARP 表（若存在则更新）
12 void arp_add(ULONG ip, const BYTE mac[6]) {
13     // 先尝试更新
14     for (int i = 0; i < MAX_ARP; ++i) {
15         if (g_arp_table[i].used && g_arp_table[i].ip == ip) {

```

```

16         memcpy(g_arp_table[i].mac, mac, 6);
17         return;
18     }
19 }
20 // 再寻找空槽插入
21 for (int i = 0; i < MAX_ARP; ++i) {
22     if (!g_arp_table[i].used) {
23         g_arp_table[i].used = 1;
24         g_arp_table[i].ip = ip;
25         memcpy(g_arp_table[i].mac, mac, 6);
26         return;
27     }
28 }
29 }

```

(2) 缓存待转发的数据报与超时丢弃

```

1 // 将一个 IP 数据报缓存起来, 等待后续 ARP 解析
2 void buffer_add(const BYTE* data, int len, ULONG dest_ip) {
3     for (int i = 0; i < MAX_BUFFER; ++i) {
4         if (!g_buffer[i].used) { // 找到空槽
5             memcpy(g_buffer[i].data, data, len);
6             g_buffer[i].len = len;
7             g_buffer[i].dest_ip = dest_ip; // 需要解析 MAC 的 IP
8             g_buffer[i].ts = clock(); // 记录入队时间
9             g_buffer[i].used = 1;
10            printf("Cache one packet, target IP: ");
11            print_ip(dest_ip); printf("\n");
12            return;
13        }
14    }
15    printf("Buffer is full, drop packet.\n");
16 }
17 // 当某 IP 的 MAC 被解析出来后, 尝试发送所有缓存的数据报
18 void buffer_try_send_for_ip(ULONG ip, const BYTE mac[6]) {
19     for (int i = 0; i < MAX_BUFFER; ++i) {
20         if (g_buffer[i].used && g_buffer[i].dest_ip == ip) {
21             send_ip_with_mac(g_buffer[i].data, g_buffer[i].len, mac);
22             g_buffer[i].used = 0; // 发送后释放缓存
23         }
24     }
25 }
26 // 周期性调用, 检查是否有缓存数据报超时
27 void buffer_check_timeout() {
28     clock_t now = clock();
29     for (int i = 0; i < MAX_BUFFER; ++i) {
30         if (g_buffer[i].used && (now - g_buffer[i].ts) >= 6000) {
31             printf("Buffer packet to ");
32             print_ip(g_buffer[i].dest_ip);
33             printf("timeout, drop.\n");
34             g_buffer[i].used = 0;
35         }
36     }
37 }

```


(3) 发送 ARP 请求：主动向下一跳 IP 询问 MAC

```

1  // 根据目的 IP 选择使用哪个本地 IP 作为 ARP 请求的源地址
2  int choose_if_for_ip(ULONG ip) {
3      for (int i = 0; i < g_ip_count; ++i) {
4          if ((ip & g_my_mask[i]) == (g_my_ip[i] & g_my_mask[i]))
5              return i;      // 选择与目的 IP 同网段的接口
6      }
7      return 0;
8  }
9  // 发送 ARP 请求，询问 target_ip 的 MAC 地址
10 void send_arp_request(ULONG target_ip) {
11     ARP_PACKET arp;
12     memset(&arp, 0, sizeof(arp));
13     // 以太网头：目的 MAC 为广播，源 MAC 为本机 MAC
14     memset(arp.eth.dest, 0xFF, 6);
15     memcpy(arp.eth.src, g_my_mac, 6);
16     arp.eth.type = htons(0x0806);      // ARP
17     // ARP 报文字段
18     arp.htype = htons(1);              // 硬件类型：以太网
19     arp.ptype = htons(0x0800);        // 协议类型：IP
20     arp.hlen = 6;
21     arp.plen = 4;
22     arp.oper = htons(1);              // 操作类型：请求
23     memcpy(arp.sha, g_my_mac, 6);     // 发送端 MAC
24     int idx = choose_if_for_ip(target_ip);
25     arp.spa = g_my_ip[idx];           // 发送端 IP
26     arp.tpa = target_ip;              // 目标 IP
27     if (pcap_sendpacket(g_adhandle, (const u_char*)&arp, sizeof(arp)) != 0)
28         printf("Send ARP request failed:\n", pcap_geterr(g_adhandle));
29     else {
30         printf("Send ARP request to ");
31         print_ip(target_ip);
32         printf(" for MAC. \n");
33     }
34 }

```

分析：

- 当 `handle_ip` 发现 ARP 表中没有下一跳 MAC 时，会调用 `buffer_add + send_arp_request`。
- 收到 ARP 应答后 `handle_arp` 会调用 `arp_add` 更新表，再调用 `buffer_try_send_for_ip` 把对应 IP 的缓存数据报全部发出去。
- `buffer_check_timeout` 在抓包循环中周期调用，用于实验中体现“缓存超时丢弃”的情况。

3.2.4 IP 数据报处理流程：获取、选路、TTL 处理与投递

设计目标：

- 只处理“发给本机 MAC”的 IP 数据报；
- 对 IP 首部做校验和验证与 TTL 递减；
- 调用路由表进行最长前缀匹配，得到下一跳 IP；
- 若已有 MAC，则直接封装以太网头并转发；否则缓存并发送 ARP。

下面是 `handle_ip` 的关键逻辑（去掉了一些边界检查，只保留核心部分）：

```
1 void handle_ip(const u_char* packet, int len, ROUTE_TABLE* rt) {
2     const ETH_HEADER* eth = (const ETH_HEADER*)packet;
3     // 1) 只处理目的 MAC 为本机的帧
4     if (memcmp(eth->dest, g_my_mac, 6) != 0)
5         return;
6     const IP_HEADER* ip = (const IP_HEADER*)(packet + sizeof(ETH_HEADER));
7     int ihl = (ip->ver_ihl & 0x0F) * 4; // IP 首部长度
8     printf("\n=== IP packet captured ===\n");
9     printf(" Src IP: "); print_ip(ip->saddr);
10    printf("\n Dst IP: "); print_ip(ip->daddr);
11    printf("\n Original TTL: %d\n", ip->ttl);
12    // 2) 验证 IP 首部校验和
13    if (!ip_header_valid((IP_HEADER*)ip)) {
14        printf(" IP header checksum error, drop.\n");
15        return;
16    }
17    // 3) 若目的 IP 为本机任一接口，则不再转发
18    for (int i = 0; i < g_ip_count; ++i) {
19        if (ip->daddr == g_my_ip[i]) {
20            printf(" Destination is local IP, not forwarded.\n");
21            return;
22        }
23    }
24    // 4) TTL <= 1 时丢弃（正常应发送 ICMP 超时报文）
25    if (ip->ttl <= 1) {
26        printf(" TTL too small, drop (could send ICMP Time Exceeded).\n");
27        return;
28    }
29    // 5) 路由查找：最长前缀匹配
30    ULONG next_hop = rt_lookup(rt, ip->daddr);
31    if (next_hop == 0xFFFFFFFF) {
32        printf(" No route found, drop.\n");
33        return;
34    }
35    printf(" Route lookup, next hop: ");
36    print_ip(next_hop);
37    printf(next_hop == ip->daddr ? " (direct delivery)\n" : " (indirect delivery)\n");
38    // 6) 拷贝一份报文，递减 TTL 并重新计算校验和
39    BYTE sendbuf[2000];
40    int sendlen = (len > (int)sizeof(sendbuf)) ? (int)sizeof(sendbuf) : len;
41    memcpy(sendbuf, packet, sendlen);
```

```

42     IP_HEADER* sip = (IP_HEADER*)(sendbuf + sizeof(ETH_HEADER));
43     sip->ttl--;           // TTL 减一
44     sip->check = 0;
45     sip->check = ip_checksum((unsigned short*)sip, ihl);
46     printf("TTL before forwarding: %d\n", sip->ttl);
47     // 7) 根据 ARP 表决定是直接转发还是缓存等待 ARP
48     BYTE dest_mac[6];
49     if (!arp_find(next_hop, dest_mac)) {
50         printf(" No MAC in ARP table, need ARP resolution.\n");
51         buffer_add(sendbuf, sendlen, next_hop); // 先缓存
52         send_arp_request(next_hop); // 发送 ARP 请求
53     } else {
54         printf(" MAC found in ARP table, forward directly.\n");
55         send_ip_with_mac(sendbuf, sendlen, dest_mac); // 直接转发
56     }
57 }

```

转发函数 `send_ip_with_mac` 负责把目的 MAC 和源 MAC 填入以太网头，然后调用 WinPcap 发送：

```

1  void send_ip_with_mac(BYTE* packet, int len, const BYTE mac[6]) { // 将目的 MAC 和源 MAC
    填入以太网头，并通过 winPcap 发送
2      ETH_HEADER* eth = (ETH_HEADER*)packet;
3      memcpy(eth->dest, mac, 6); // 下一跳 MAC
4      memcpy(eth->src, g_my_mac, 6); // 本机 MAC
5      eth->type = htons(0x0800); // IP 帧
6      if (pcap_sendpacket(g_adhandle, packet, len) != 0) {
7          printf("Send IP packet failed: %s\n", pcap_geterr(g_adhandle));
8      } else {
9          printf("Forward one packet, dest MAC: ");
10         for (int i = 0; i < 6; ++i) {
11             printf("%02x", mac[i]);
12             if (i < 5) printf("-");
13         }
14         printf("\n");
15     }
16 }

```

分析：

- 这一段完整体现了“获取 IP 数据报 → 选路 → 投递”的全过程，是实验的核心。
- 日志中会打印源/目的 IP、TTL 变化、下一跳 IP、是否直连、是否命中 ARP 表等，和指导书中“显示数据报获取和转发过程”的要求一一对应。

3.2.5 主控流程与菜单：路由表手工插入 / 删除

设计目标：

- 在 `main` 中完成适配器选择、WinPcap 打开、直连路由初始化、抓包线程创建；
- 通过命令行菜单实现“手工插入、删除路由表项”和“显示路由表”，直接对应实验要求第 (3) 点。

菜单部分的关键代码如下：

```

1  while (1) {
2      int op;
3      printf("\n===== MENU =====\n");
4      printf(" 1. Add route\n");
5      printf(" 2. Delete route\n");
6      printf(" 3. Show route table\n");
7      printf(" 4. Exit\n");
8      printf("Input choice: ");
9      scanf("%d", &op);
10     if (op == 1) {          // 手工插入路由
11         char buf[32];
12         ULONG netmask, destnet, nexthop;
13         printf("Input netmask(e.g. 255.255.255.0):\n");//请输入子网掩码(如
255.255.255.0)
14         scanf("%s", buf);
15         netmask = inet_addr(buf);
16         printf("Input destination network (e.g. 206.1.3.0):\n");//请输入目的网络(如
206.1.3.0)
17         scanf("%s", buf);
18         destnet = inet_addr(buf);
19         printf("Input next hop IP (e.g. 206.1.2.2):\n");//请输入下一跳 IP(如 206.1.2.2)
20         scanf("%s", buf);
21         nexthop = inet_addr(buf);
22         rt_add(&rtable, netmask, destnet, nexthop, 1); // type=1 静态路由
23     }
24     else if (op == 2) { // 手工删除路由
25         int idx;
26         printf("Input index to delete (start from 1):\n");//请输入要删除的序号(从 1 开始)
27         scanf("%d", &idx);
28         rt_delete(&rtable, idx - 1);
29     }
30     else if (op == 3) { // 查看路由表
31         rt_print(&rtable);
32     }
33     else if (op == 4) {
34         printf("Exit program...\n");
35         break;          // 退出程序
36     }
37 }

```

分析:

- 添加静态路由时，路由表函数 `rt_add` 会负责把它插入到正确位置，保证最长前缀匹配。
- 删除时，`rt_delete` 会阻止删除类型为 0 的直连路由，只允许删手工添加的静态路由，既安全又符合实验要求。
- 通过在实验中多次执行“添加→显示→删除→再显示”，可以直观地展示路由表的变化情况，为后面的转发实验和报告截图提供依据。

3.3 与现有路由器协同工作的静态路由配置实验

3.3.1 启动虚拟机并检查 IP 配置

- 1. 在虚拟机管理器中依次启动四台虚拟机。
- 2. 分别登录虚拟机 1、2、3、4，在每台虚拟机的命令提示符中执行：

```
1 | ipconfig
```

检查并确认各主机 IP 配置符合指导书要求：

- VM1：206.1.1.2 / 255.255.255.0，网关 206.1.1.1

```
Connection-specific DNS Suffix . :  
IP Address. . . . . : 206.1.1.2  
Subnet Mask . . . . . : 255.255.255.0  
Default Gateway . . . . . : 206.1.1.1
```

- VM2：同一块网卡上有两个 IP：206.1.1.1 /24 和 206.1.2.1 /24

```
Connection-specific DNS Suffix . :  
IP Address. . . . . : 206.1.2.1  
Subnet Mask . . . . . : 255.255.255.0  
IP Address. . . . . : 206.1.1.1  
Subnet Mask . . . . . : 255.255.255.0  
Default Gateway . . . . . :
```

- VM3：206.1.2.2 /24 与 206.1.3.1 /24，充当路由器 B

```
Connection-specific DNS Suffix . :  
IP Address. . . . . : 206.1.3.1  
Subnet Mask . . . . . : 255.255.255.0  
IP Address. . . . . : 206.1.2.2  
Subnet Mask . . . . . : 255.255.255.0  
Default Gateway . . . . . :
```

- VM4：206.1.3.2 /24，网关 206.1.3.1

```
Connection-specific DNS Suffix . :  
IP Address. . . . . : 206.1.3.2  
Subnet Mask . . . . . : 255.255.255.0  
Default Gateway . . . . . : 206.1.3.1
```

3.3.2 在路由器 B（虚拟机 3）上配置静态路由

- 1. 在虚拟机 3 中以管理员身份打开命令提示符。
- 2. 先查看当前路由表，确认系统默认路由情况：

```
1 | route print
```

- 3. 为了让路由器 B 能把发往 206.1.1.0/24 的报文转发回软件路由器（VM2），在 VM3 上执行下面的静态路由命令：

```
1 | route add 206.1.1.0 mask 255.255.255.0 206.1.2.1
```

其中：

- 206.1.1.0 是目标网络地址（VM1 所在网段）；
- mask 255.255.255.0 是子网掩码；

- 206.1.2.1 是下一跳地址，即虚拟机 2 上软件路由器的接口 IP。

4. 再次使用 `route print` 查看路由表，可以看到新增了一条

1	Network Destination	Netmask	Gateway
2	206.1.1.0	255.255.255.0	206.1.2.1

的条目。

```

命令提示符

IPv4 Route Table
=====
Interface List
0x1 ..... MS TCP Loopback interface
0x10003 ...00 0c 29 a1 0b de ..... Intel(R) PRO/1000 MT Network Connection
=====
Active Routes:
Network Destination        Netmask          Gateway           Interface        Metric
-----
127.0.0.0                  255.0.0.0        127.0.0.1         127.0.0.1         1
206.1.1.0                  255.255.255.0    206.1.2.1         206.1.2.2         1
206.1.2.0                  255.255.255.0    206.1.2.2         206.1.2.2         10
206.1.2.2                  255.255.255.255  127.0.0.1         127.0.0.1         10
206.1.2.255               255.255.255.255  206.1.2.2         206.1.2.2         10
206.1.3.0                  255.255.255.0    206.1.3.1         206.1.2.2         10
206.1.3.1                  255.255.255.255  127.0.0.1         127.0.0.1         10
206.1.3.255               255.255.255.255  206.1.3.1         206.1.2.2         10
224.0.0.0                  240.0.0.0        206.1.2.2         206.1.2.2         10
255.255.255.255           255.255.255.255  206.1.2.2         206.1.2.2         1
=====
Persistent Routes:
None

C:\Documents and Settings\Administrator>
  
```

至此，路由器 B 已经知道“如何到达 206.1.1.0/24”。

3.3.3 在软件路由器（虚拟机 2）上启动程序并选择接口

1. 将宿主机生成的 `ROUTER\Release\ROUTER.exe` 拷贝到虚拟机 2 的桌面，并双击启动。
2. 程序启动后，会通过 WinPcap 列出所有可用的网络适配器及其 IP/MASK 信息：

```

C:\Documents and Settings\Administrator\桌面\ROUTER.exe

Available adapters:
1. \Device\NPF_{3AC00148-1BFF-47AF-8441-F1D14A619031} Desc: Intel(R) PRO/1000 M
T Network Connection
   IP: 206.1.2.1  MASK: 206.1.2.1
   IP: 206.1.1.1  MASK: 206.1.1.1
Please select adapter index: 1
  
```

从列表中找到同时具有 206.1.1.1 和 206.1.2.1 的那块网卡，输入其序号（1）并回车。

3. 选定网卡后，程序首先构造 ARP 请求获取本机 MAC 地址，然后根据两个 IP 及其掩码自动生成两条直连路由，并打印初始路由表：

```
C:\Documents and Settings\Administrator\桌面\ROUTER.exe
Desc: Intel(R) PRO/1000 MT Network Connection
Local IP1: 206.1.2.1 MASK: 255.255.255.0
Local IP2: 206.1.1.1 MASK: 255.255.255.0
Send ARP to get local MAC...
Local MAC: 00-0C-29-79-AE-AA

Initial route table:
<===== Routing Table =====>
Entry 11 Type: Direct
Destination network: 206.1.2.0
Netmask: 255.255.255.0
Next hop IP: <Direct>
-----
Entry 21 Type: Direct
Destination network: 206.1.1.0
Netmask: 255.255.255.0
Next hop IP: <Direct>

===== MENU =====
1. Add route
2. Delete route
3. Show route table
4. Exit
Input choice: _
```

这说明软件路由器已经自动知道：

- 如何到达 206.1.1.0/24（通过本机接口 206.1.1.1 直连）；
- 如何到达 206.1.2.0/24（通过本机接口 206.1.2.1 直连）。

3.3.4 协同工作原理与路径说明

在完成以上静态路由配置后，软件路由器（VM2）与路由器 B（VM3）之间的协同关系如下：

1. 从 206.1.1.0/24 到 206.1.3.0/24：

- VM1（206.1.1.2）将默认网关设置为 206.1.1.1，因此发往 206.1.3.2 的 IP 数据报首先被送到软件路由器 VM2；
- VM2 路由表中有直连路由 206.1.2.0/24，因此对目的 206.1.3.x 的报文执行最长前缀匹配时，会根据后续添加的静态路由（在 3.4 中专门讨论）将其转发到下一跳 206.1.2.2，即 VM3；
- VM3 根据前面配置的静态路由 206.1.1.0/24 via 206.1.2.1 和自己的直连网络 206.1.3.0/24，可以完成报文在 206.1.2.0/24 与 206.1.3.0/24 之间的转发；
- 最终，数据报到达 VM4（206.1.3.2），实现端到端通信。

2. 从 206.1.3.0/24 返回 206.1.1.0/24：

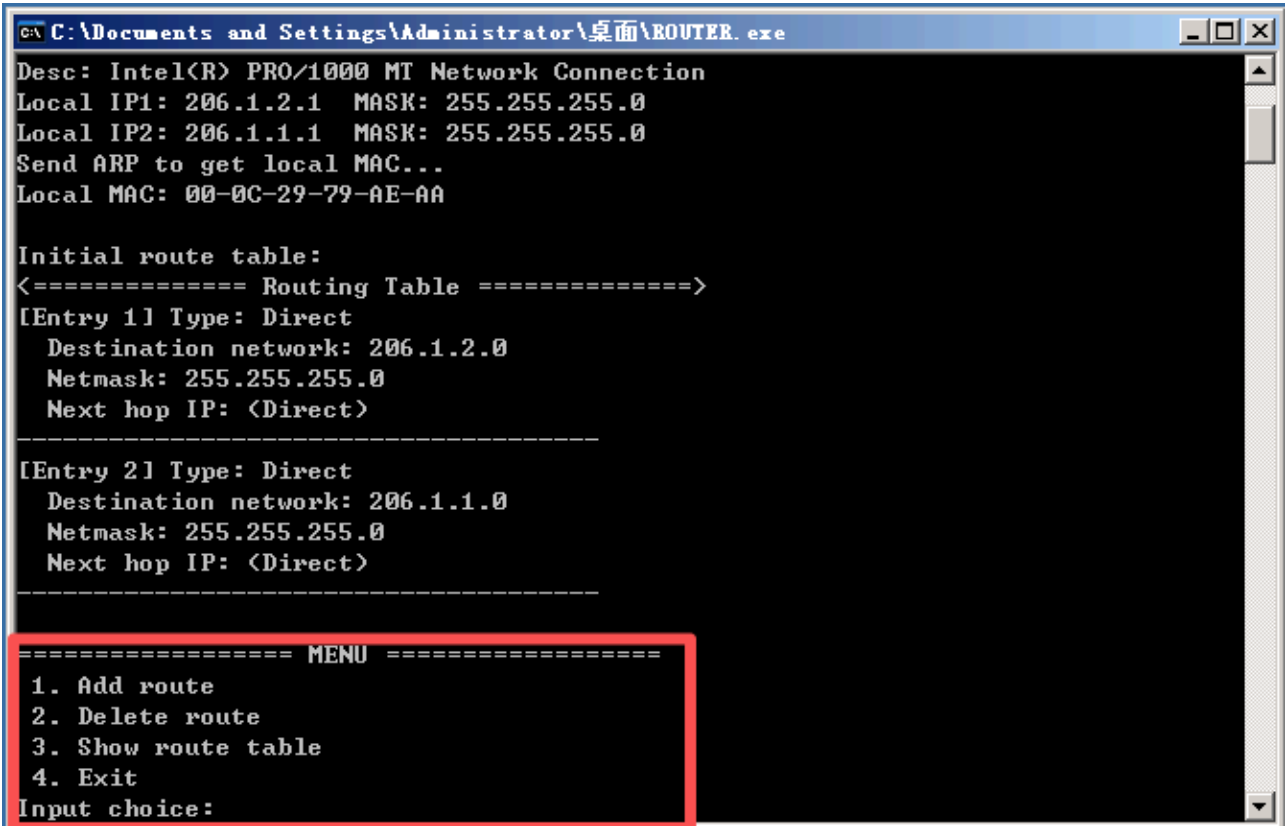
- VM4 将默认网关设置为 206.1.3.1，发往 206.1.1.2 的报文先到达 VM3；
- VM3 发现目的地址属于 206.1.1.0/24，匹配到刚添加的静态路由，下一跳为 206.1.2.1，于是将报文转发给软件路由器 VM2；
- VM2 再根据自己的直连路由 206.1.1.0/24，把报文直接投递给 VM1（206.1.1.2）。

通过上述配置，现有硬件路由器（VM3）和软件路由器（VM2）共同构成了一条从 206.1.1.0/24 到 206.1.3.0/24 的完整路径，满足实验要求中“完成的路由器程序能和现有路由器产品协同工作”的功能。后续 3.4 与 3.5 中的路由表维护和转发日志分析，均是在此基础上进行的。

3.4 路由表手工插入与删除功能实验

3.4.1 实验前准备：确认初始路由表

1. 在虚拟机 2 中运行 `ROUTER.exe`，选择带有 `206.1.1.1` 和 `206.1.2.1` 的那块网卡（步骤同 3.3）。
2. 程序启动后会自动插入两条直连路由，并打印初始路由表，菜单界面如下：



```
C:\Documents and Settings\Administrator\桌面\ROUTER.exe
Desc: Intel(R) PRO/1000 MT Network Connection
Local IP1: 206.1.2.1 MASK: 255.255.255.0
Local IP2: 206.1.1.1 MASK: 255.255.255.0
Send ARP to get local MAC...
Local MAC: 00-0C-29-79-AE-AA

Initial route table:
<===== Routing Table =====>
[Entry 1] Type: Direct
  Destination network: 206.1.2.0
  Netmask: 255.255.255.0
  Next hop IP: <Direct>
-----
[Entry 2] Type: Direct
  Destination network: 206.1.1.0
  Netmask: 255.255.255.0
  Next hop IP: <Direct>
-----

===== MENU =====
1. Add route
2. Delete route
3. Show route table
4. Exit
Input choice:
```

之后所有操作都通过此菜单完成。

3.4.2 使用菜单添加静态路由 206.1.3.0/24

目标：让软件路由器知道“到 206.1.3.0/24 网段的下一跳是 206.1.2.2（虚拟机 3）”。

1. 在菜单提示符 `Input choice:` 下输入 `1`，选择 **Add route**，程序进入添加路由模式。
2. 程序依次提示输入三个字段：

1) 输入子网掩码 `Input netmask (e.g. 255.255.255.0):`

1

255.255.255.0

对应 `206.1.3.0/24` 网段。

2) 输入目的网络地址 `Input destination network (e.g. 206.1.3.0):`

1

206.1.3.0

表示要为 206.1.3.0 网段添加一条路由。

3) 输入下一跳 IP 地址 `Input next hop IP (e.g. 206.1.2.2):`

即路由器 B 在 206.1.2.0/24 网段上的接口地址。

3. 输入完成后，程序调用 `rt_add()` 将该条静态路由插入链表，并输出提示 Add OK.：

```

C:\Documents and Settings\Administrator\桌面\ROUTER.exe
Destination network: 206.1.1.0
Netmask: 255.255.255.0
Next hop IP: <Direct>
-----
===== MENU =====
1. Add route
2. Delete route
3. Show route table
4. Exit
Input choice: 1
Input netmask (e.g. 255.255.255.0):
255.255.255.0
Input destination network (e.g. 206.1.3.0):
206.1.3.0
Input next hop IP (e.g. 206.1.2.2):
206.1.2.2
Add OK.
===== MENU =====
1. Add route
2. Delete route
3. Show route table
4. Exit
Input choice:

```

4. 为验证添加结果，在菜单中输入 3 选择 Show route table，程序输出当前路由表：

```

C:\Documents and Settings\Administrator\桌面\ROUTER.exe
4. Exit
Input choice: 3
<===== Routing Table =====>
[Entry 1] Type: Direct
Destination network: 206.1.2.0
Netmask: 255.255.255.0
Next hop IP: <Direct>
-----
[Entry 2] Type: Direct
Destination network: 206.1.1.0
Netmask: 255.255.255.0
Next hop IP: <Direct>
-----
[Entry 3] Type: Static
Destination network: 206.1.3.0
Netmask: 255.255.255.0
Next hop IP: 206.1.2.2
-----

```

说明：

- 前两条是程序根据本机 IP 自动生成的直连路由；
- 第三条是本节手工添加的静态路由，类型为 `Static`，下一跳为 `206.1.2.2`；
- 之后 `rt_lookup()` 对目的地址为 `206.1.3.x` 的数据报进行最长前缀匹配时，会先命中这条静态路由，从而把数据报转发给路由器 B。

3.4.3 验证路由表删除功能与直连路由保护

本小节分别测试：① 删除静态路由；② 尝试删除直连路由时程序的保护机制。

(1) 删除静态路由条目

1. 确保当前路由表中已有三条路由（两条 Direct + 一条 Static）。

```
Input choice: 0
<===== Routing Table =====>
[Entry 1] Type: Direct
  Destination network: 206.1.2.0
  Netmask: 255.255.255.0
  Next hop IP: <Direct>
-----
[Entry 2] Type: Direct
  Destination network: 206.1.1.0
  Netmask: 255.255.255.0
  Next hop IP: <Direct>
-----
[Entry 3] Type: Static
  Destination network: 206.1.3.0
  Netmask: 255.255.255.0
  Next hop IP: 206.1.2.2
-----
```

2. 在菜单中输入 **2**，选择 **Delete route**。
3. 程序提示输入要删除的序号（从 1 开始）：

1 | Input index to delete (start from 1):

4. 这里输入 **3**，表示删除第 3 条静态路由（206.1.3.0/24）。程序调用 `rt_delete()` 删除该项并输出删除结果。
5. 再次在菜单中输入 **3** 查看路由表：

```
C:\Documents and Settings\Administrator\桌面\ROUTER.exe
1. Add route
2. Delete route
3. Show route table
4. Exit
Input choice: 2
Input index to delete <start from 1>:
3

===== MENU =====
1. Add route
2. Delete route
3. Show route table
4. Exit
Input choice: 3
<===== Routing Table =====>
[Entry 1] Type: Direct
  Destination network: 206.1.2.0
  Netmask: 255.255.255.0
  Next hop IP: <Direct>
-----
[Entry 2] Type: Direct
  Destination network: 206.1.1.0
  Netmask: 255.255.255.0
  Next hop IP: <Direct>
-----
```

可以看到静态路由已被成功移除。

(2) 尝试删除直连路由（验证保护机制）

1. 再次进入菜单 2：输入 2，选择 Delete route。
2. 当程序提示 Input index to delete (start from 1): 时，输入 1 或 2，尝试删除第 1 或第 2 条直连路由。
3. 由于在 rt_delete() 中对 type==0 的条目做了保护，程序会打印提示信息 Direct/default route cannot be deleted.:

```
===== MENU =====
1. Add route
2. Delete route
3. Show route table
4. Exit
Input choice: 2
Input index to delete (start from 1):
1
Direct/default route cannot be deleted.

===== MENU =====
1. Add route
2. Delete route
3. Show route table
4. Exit
Input choice: 2
Input index to delete (start from 1):
2
Direct/default route cannot be deleted.
```

4. 此时使用菜单 3 查看路由表，可以看到两条直连路由仍然存在，没有被删除。

```
===== MENU =====
1. Add route
2. Delete route
3. Show route table
4. Exit
Input choice: 3
<===== Routing Table =====>
[Entry 1] Type: Direct
Destination network: 206.1.2.0
Netmask: 255.255.255.0
Next hop IP: (Direct)
-----
[Entry 2] Type: Direct
Destination network: 206.1.1.0
Netmask: 255.255.255.0
Next hop IP: (Direct)
```

说明：

- 通过删除第 3 条静态路由，可以证明程序实现了“手工删除路由表项”的功能；
- 尝试删除直连路由时，程序检测到 type==0，拒绝删除并给出提示，实现了对直连/缺省路由的保护，避免错误操作导致本机网络不可达。

3.4.4 手工路由配置对转发的影响（现象概述）

为了体现“添加/删除静态路由对转发结果的影响”，本实验还做了简单对比（详细日志分析见 3.5）：

1. 未添加静态路由时：

- 在 VM1 上执行 `ping 206.1.3.2`，软件路由器 VM2 的日志中会显示
`No route found, drop.`，说明路由表中没有到 206.1.3.0/24 的路由，转发失败。

```
C:\Documents and Settings\Administrator>ping 206.1.3.2

Pinging 206.1.3.2 with 32 bytes of data:

Request timed out.
Request timed out.
Request timed out.
Request timed out.

Ping statistics for 206.1.3.2:
    Packets: Sent = 4, Received = 0, Lost = 4 (100% loss),

C:\Documents and Settings\Administrator>
```

```
=== IP packet captured ===
Src IP: 206.1.1.2
Dst IP: 206.1.3.2
Original TTL: 128
No route found, drop.
```

2. 添加静态路由 206.1.3.0/24 → 206.1.2.2 后：

- 再次在 VM1 上 `ping 206.1.3.2`，VM2 的日志显示
`Route lookup, next hop: 206.1.2.2 (indirect delivery)`，并进行 ARP 解析和数据转发。

```
C:\Documents and Settings\Administrator\桌面\ROUTER.exe

=== ARP packet captured ===
Sender IP: 206.1.1.2 Target IP: 206.1.1.1
ARP request.

=== ARP packet captured ===
Sender IP: 206.1.1.1 Target IP: 206.1.1.2
ARP reply.
Sender MAC: 00-0C-29-79-AE-AA

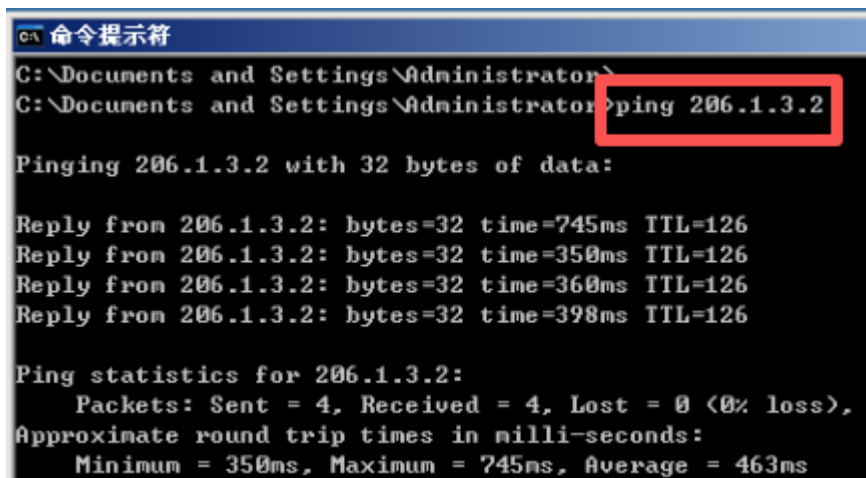
=== IP packet captured ===
Src IP: 206.1.1.2
Dst IP: 206.1.3.2
Original TTL: 128
Route lookup, next hop: 206.1.2.2 (indirect delivery)
TTL before forwarding: 127
No MAC in ARP table, need ARP resolution.
Cache one packet, target IP: 206.1.2.2
Send ARP request to 206.1.2.2 for MAC.

=== ARP packet captured ===
Sender IP: 206.1.2.1 Target IP: 206.1.2.2
ARP request.

=== ARP packet captured ===
Sender IP: 206.1.2.2 Target IP: 206.1.2.1
```

- 206.1.2.1 -> 206.1.2.2 的 ARP request
- 206.1.2.2 -> 206.1.2.1 的 ARP reply

说明路由器先根据路由表算出下一跳 206.1.2.2，再发 ARP 去解析它的 MAC，这就是“进行 ARP 解析和数据转发”的过程。



```
C:\Documents and Settings\Administrator\桌面>ping 206.1.3.2

Pinging 206.1.3.2 with 32 bytes of data:

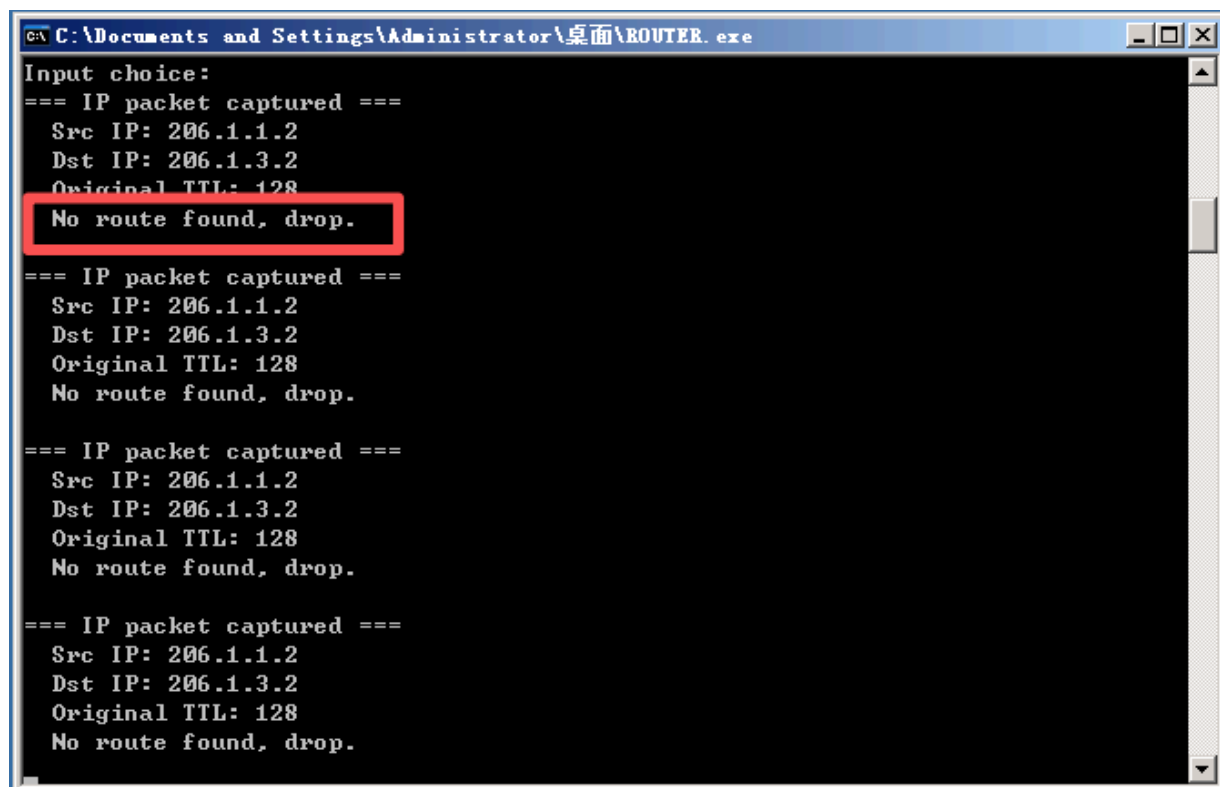
Reply from 206.1.3.2: bytes=32 time=745ms TTL=126
Reply from 206.1.3.2: bytes=32 time=350ms TTL=126
Reply from 206.1.3.2: bytes=32 time=360ms TTL=126
Reply from 206.1.3.2: bytes=32 time=398ms TTL=126

Ping statistics for 206.1.3.2:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 350ms, Maximum = 745ms, Average = 463ms
```

最终 ping 成功。

3. 删除静态路由后：

- 使用菜单 2 删除第 3 条静态路由，再次 ping 206.1.3.2，日志中又重新出现 No route found, drop.，说明静态路由删除立即生效。



```
C:\Documents and Settings\Administrator\桌面\ROUTER.exe

Input choice:
=== IP packet captured ===
Src IP: 206.1.1.2
Dst IP: 206.1.3.2
Original TTL: 128
No route found, drop.

=== IP packet captured ===
Src IP: 206.1.1.2
Dst IP: 206.1.3.2
Original TTL: 128
No route found, drop.

=== IP packet captured ===
Src IP: 206.1.1.2
Dst IP: 206.1.3.2
Original TTL: 128
No route found, drop.

=== IP packet captured ===
Src IP: 206.1.1.2
Dst IP: 206.1.3.2
Original TTL: 128
No route found, drop.
```

这些现象表明：软件路由器的路由表可以通过菜单进行手工插入和删除，并且路由表的变化会直接影响后续 IP 数据报的转发行为，完全符合实验中对静态路由配置功能的要求。

3.5 IP 数据报获取、选路和投递及工作日志分析

本节在前面已经配置好虚拟网络和静态路由的基础上，通过在 VM1(206.1.1.2) 上 ping 206.1.3.2，观察软件路由器 VM2(206.1.1.1/206.1.2.1) 控制台的日志输出，分析 IP 数据报在本实验拓扑中的转发过程。

3.5.1 未配置静态路由时的现象

在 VM2 只保留两条直连路由（206.1.1.0/24 和 206.1.2.0/24），未添加任何静态路由时，在 VM1 上执行：

```
1 | ping 206.1.3.2
```

此时 VM2 的日志如图 3-5-1 所示，每次收到从 206.1.1.2 到 206.1.3.2 的 IP 数据报后，只输出：

```
=== IP packet captured ===  
Src IP: 206.1.1.2  
Dst IP: 206.1.3.2  
Original TTL: 100  
No route found, drop.
```

分析如下：

- 软件路由器已经正确捕获到了 IP 数据报，源地址和目的地址与 ping 命令一致。
- 由于路由表中只包含两个直连网络 206.1.1.0/24、206.1.2.0/24，没有任何到 206.1.3.0/24 的条目，`rt_lookup()` 查表失败，于是输出 `No route found, drop.` 并丢弃该报文。
- VM1 端看到的现象就是 ping 全部超时。

这一部分验证了：如果没有正确的路由项，即使抓到了数据报，路由器也不会转发。

3.5.2 配置静态路由后第一次 ping：ARP 解析与缓存

在菜单中添加静态路由：

```
1 | Dest network : 206.1.3.0  
2 | Netmask      : 255.255.255.0  
3 | Next hop     : 206.1.2.2
```

此时 VM2 的路由表中多了一条“到 206.1.3.0/24 经 206.1.2.2”的静态路由。再次在 VM1 上执行 `ping 206.1.3.2`，VM2 日志上半部分关键输出如下：

```
=== ARP packet captured ===  
Sender IP: 206.1.1.2 Target IP: 206.1.1.1  
ARP request.  
  
=== ARP packet captured ===  
Sender IP: 206.1.1.1 Target IP: 206.1.1.2  
ARP reply.  
Sender MAC: 00-0C-29-79-AE-AA
```

- 首先，VM1 为了把报文发给默认网关 206.1.1.1，先对其发起 ARP 请求；
- 软件路由器回复 ARP 应答，日志中打印出本机 MAC 地址；
- 说明 链路层 ARP 解析正常，本机 MAC 已加入对端主机的 ARP 表。

随后，软件路由器捕获到真正需要转发的 IP 数据报：

```
=== IP packet captured ===
Src IP: 206.1.1.2
Dst IP: 206.1.3.2
Original TTL: 128
Route lookup, next hop: 206.1.2.2 <indirect delivery>
TTL before forwarding: 127
No MAC in ARP table, need ARP resolution.
Cache one packet, target IP: 206.1.2.2
Send ARP request to 206.1.2.2 for MAC.
```

逐行分析：

1. 程序识别出源 IP 为 206.1.1.2、目的 IP 为 206.1.3.2，说明IP 数据报已被成功解析。
2. Route lookup, next hop: 206.1.2.2 (indirect delivery) 表明：
 - 通过 `rt_lookup()` 在路由表中找到静态路由 206.1.3.0/24 via 206.1.2.2；
 - 当前转发采用的是间接投递，下一跳为路由器 B 在 206.1.2.0/24 上的地址 206.1.2.2。
3. TTL before forwarding: 127 说明程序在转发前对 IP 首部执行了 `TTL--` 并重新计算校验和，符合路由器基本功能。
4. 由于这是第一次访问 206.1.2.2，ARP 表中尚无其 MAC，日志输出
`No MAC in ARP table, need ARP resolution.`，并调用 `buffer_add()` 将该数据报缓存：
`Cache one packet, target IP: 206.1.2.2。`
5. 最后调用 `send_arp_request()` 向 206.1.2.2 发 ARP 请求：
`Send ARP request to 206.1.2.2 for MAC.`

紧接着，VM2 又依次输出：

```
=== ARP packet captured ===
Sender IP: 206.1.2.1 Target IP: 206.1.2.2
ARP request.

=== ARP packet captured ===
Sender IP: 206.1.2.2 Target IP: 206.1.2.1
ARP reply.
Sender MAC: 00-0C-29-A1-0B-DE
Forward one packet, dest MAC: 00-0C-29-A1-0B-DE
```

这表明：

- VM2 在 206.1.2.0/24 网段上成功发出了 ARP 请求；
- VM3 回复 ARP 应答，软件路由器在 `handle_arp()` 中更新了 ARP 表，并调用 `buffer_try_send_for_ip()` 将刚才缓存的 ICMP 请求报文重新封装后发出；
- `Forward one packet, dest MAC: ...` 就是基于新解析出的下一跳 MAC 地址完成的真正转发动作。

至此，从 206.1.1.2 发往 206.1.3.2 的第一个回显请求已经通过两台路由器成功到达目标网段。

3.5.3 返回方向报文与后续快速转发

当 VM4(206.1.3.2) 收到 ICMP 请求后，会向源地址 206.1.1.2 返回 ICMP 回显应答。VM2 的日志中可以看到一系列从 206.1.3.2 到 206.1.1.2 的 IP 报文：

```
=== IP packet captured ===
Src IP: 206.1.3.2
Dst IP: 206.1.1.2
Original TTL: 127
Route lookup, next hop: 206.1.1.2 <direct delivery>
TTL before forwarding: 126
MAC found in ARP table, forward directly.
Forward one packet, dest MAC: 00-0C-29-4F-51-52
```

分析：

- 返回方向的源 IP 为 206.1.3.2，目的 IP 为 206.1.1.2；
- 对于目的网段 206.1.1.0/24，VM2 的路由表中有直连路由，因此 `next hop` 直接就是 206.1.1.2，“direct delivery”；
- 之前 ARP 交互已经让 VM2 获得了 206.1.1.2 的 MAC，故日志显示 `MAC found in ARP table, forward directly.`；
- `Forward one packet, dest MAC: ...` 表示路由器已将 ICMP 应答直接发回给 VM1。

同样，在之后的几个 ping 报文（第二个、第三个、第四个请求/应答）中，可以看到日志反复出现如下模式：

```
=== IP packet captured ===
Src IP: 206.1.1.2
Dst IP: 206.1.3.2
Original TTL: 128
Route lookup, next hop: 206.1.2.2 <indirect delivery>
TTL before forwarding: 127
MAC found in ARP table, forward directly.
Forward one packet, dest MAC: 00-0C-29-A1-0B-DE

=== IP packet captured ===
Src IP: 206.1.3.2
Dst IP: 206.1.1.2
Original TTL: 127
Route lookup, next hop: 206.1.1.2 <direct delivery>
TTL before forwarding: 126
MAC found in ARP table, forward directly.
Forward one packet, dest MAC: 00-0C-29-4F-51-52

=== IP packet captured ===
Src IP: 206.1.1.2
Dst IP: 206.1.3.2
Original TTL: 128
Route lookup, next hop: 206.1.2.2 <indirect delivery>
TTL before forwarding: 127
MAC found in ARP table, forward directly.
Forward one packet, dest MAC: 00-0C-29-A1-0B-DE
```

说明此时：

- ARP 表中已经有了 206.1.2.2 和 206.1.1.2 的 MAC 映射；
- 路由器不再需要缓存和发送 ARP 请求，而是**直接查表转发**，实现快速转发；
- 在 VM1 的命令窗口中，对应的就是四次 ping 都能收到成功的 Reply。

3.6 关键功能扩展与细节优化

在完成指导书要求的“获取 IP 数据报、选路、投递、手工维护路由表和输出工作日志”基础上，本实验的软件路由器在实现上还做了多处细节优化和扩展：

1. 路由表按类型与掩码排序，天然支持最长前缀匹配

- 路由表使用链表结构存储，所有直连路由(type=0) 固定放在表头区域，其后才是静态路由(type=1)。
- 对静态路由采用“按掩码从大到小插入”的方式维护顺序，查表时只需从头顺序遍历，第一个匹配到的路由就是前缀最长的那一条，无需额外比较掩码长度。
- 同时在 `rt_delete()` 中对 `type==0` 的条目加了保护，输出“默认/直连路由不可删除！”，避免误删直连路由导致网络完全不可达。

2. ARP 表 + 报文缓存 + 超时丢弃机制

- 程序不只是简单查 ARP 表，而是实现了一个完整的缓存机制：
 - 当 ARP 表中没有下一跳的 MAC 时，将 IP 数据报复制到 `g_buffer` 队列中，记录目的 IP 和入队时间，并打印“缓存一个待转发数据包，目标 IP: ...”；
 - 同时主动发送 ARP 请求，请求成功后调用 `buffer_try_send_for_ip()` 对所有等待该 IP 的报文进行补 MAC 并转发；
 - 在抓包主循环中周期调用 `buffer_check_timeout()`，超过 6 秒的缓存报文会被丢弃并输出“超时丢弃”，防止缓存无限堆积。
- 这一套机制很好地模拟了真实路由器中“ARP 未完成时暂存报文”的行为，同时保证缓存不会无限增长。

3. 多网段接口的自适应选择

- 本实验在同一块虚拟网卡上配置了两个 IP（206.1.1.1 和 206.1.2.1），程序通过 `choose_if_for_ip()` 函数，根据
`目标 IP & 本机掩码 == 本机 IP & 掩码`
的规则来判断应该使用哪一个本地地址作为 ARP 源 IP。
- 这样在 206.1.1.0/24 与 206.1.2.0/24 两个网段之间转发时，ARP 请求总能从正确的网段发出，保证了多网段场景下的正确性。

4. 通过“伪造 ARP”自动获取本机 MAC

- 程序没有直接调用复杂的系统 API，而是采用了一个很巧妙的方法：
 - 构造一个发往自己 IP 地址的 ARP 请求帧（源/目的 IP 都是本机 IP）；
 - 发送后等待 WinPcap 抓到操作系统回给自己的 ARP 应答，从应答报文中提取本机 MAC，填入 `g_my_mac`。
- 这种方式与具体操作系统版本、网卡驱动关系较小，逻辑直观，也方便在实验报告中说明“如何获取本机 MAC 地址”。

5. 严格的 IP 首部校验和与 TTL 检查

- 在 `handle_ip()` 中先调用 `ip_header_valid()` 对 IP 首部校验和进行验证，错误则直接丢弃并打印“IP 首部校验和错误，丢弃”；
- 对需要转发的所有 IP 数据报，统一在拷贝后的首部里执行 `TTL--`，然后重新计算首部校验和；当 `TTL <= 1` 时输出“TTL 太小，丢弃（可以扩展为发送 ICMP Time Exceeded）”，为后续扩展 ICMP 功能预留了位置。
- 这些处理使得程序更加接近真实路由器的行为，而不只是“简单改 MAC 就发出去”。

6. 细粒度的工作日志，便于教学和调试

- 每捕获一条 ARP/IP 报文都会打印清晰的分隔标记，例如 `=== 捕获到 ARP 报文 ===`、`=== 捕获到 IP 数据报 ===`，并给出源/目的 IP、报文类型等信息；
- 在选路阶段会输出“路由查找结果，下一跳 IP: ...（直接投递/间接投递）”；
- ARP 命中/未命中、缓存入队、缓存超时、转发成功/失败都会有对应提示。
- 这些日志直接体现了“获取 → 选路 → 投递 → 结果”的完整链路，不仅方便自己调试，也方便在课堂展示和截图写报告时说明每一步发生了什么。

本实验在完成基本功能的同时，对路由表组织、ARP 协同、多网段支持、MAC 获取方式以及日志可视化等方面做了较为细致的实现和优化，使得软件路由器的行为更加接近实际产品，并且具有较好的可观察性和可扩展性。

四、实验总结

4.1 可以改进的方向

1. 补全 ICMP 差错报文功能

目前在 TTL 过期、无路由等情况下，程序只是简单打印日志并丢弃数据报（例如 `TTL too small, drop.`、`No route found, drop.`）。

后续可以在这些分支中构造 ICMP “Time Exceeded”、“Destination Unreachable”等差错报文返回给源主机，这样行为会更接近真实路由器，也便于上层应用快速定位故障。

2. 支持更多路由协议与路由表类型

本实验只实现了直连路由和手工静态路由，路由表内容完全由人工维护。

可以在现有路由表结构的基础上扩展：

- 增加默认路由（0.0.0.0/0）；
- 尝试实现简单的 RIP/OSPF 交换，自动学习路由；
- 为路由条目增加优先级、度量值等字段，支持多条路由的选择与备份。

3. 完善多接口与多线程性能

当前版本假设使用同一物理网卡上的两个 IP 地址，内部只维护一个捕获线程。可以进一步改进：

- 支持多块网卡，每个接口维护独立的 IP/MAC 与路由条目；
- 针对不同接口创建独立抓包线程，并在转发表中增加“出接口”字段；
- 在高并发场景下引入锁或无锁队列，提高缓存队列和 ARP 表访问的并发安全性和性能。

4. 增强健壮性与异常处理

- 对缓存队列满、内存分配失败、pcap 发送失败等情况，目前多为简单打印提示，后续可以进行重试或退避处理；
- 对非法输入（例如手动添加路由时掩码或 IP 写错）做更严格的校验，避免生成无效路由；
- 增加配置文件或命令行参数，自动加载常用路由而无需每次手动输入。

5. 友好性与可视化改进

- 为路由表、ARP 表和缓存队列提供更表格化的打印格式，便于截图与分析；
 - 进一步将关键日志与统计信息（转发次数、丢包原因、ARP 命中率等）汇总输出，甚至可以做一个简单的图形界面或 Web 管理页面，展示路由器运行状态。
-

4.2 总结与心得

总体来看，这次实验不仅让我把课堂上关于路由器、ARP、TTL 等知识真正“跑起来”，也训练了自己分析日志、阅读报文和排查故障的能力。后续如果有机会扩展 ICMP 差错报文或简单动态路由协议，相信在现有代码框架上继续深化，会对理解真实网络设备的实现有更大的帮助。